Applied Reinforcement Learning

Dynamic Programming

Riccardo Berta

2026.10.01

Contents

1 Dynamic Programming **1**
1.1 Value functions . . . 2
1.1.1 State-value function . . . 2
1.1.2 Action-value function . . . 4
1.2 Optimal Policies . . . 4
1.2.1 Policy Evaluation . . . 6
1.2.2 Policy Improvement . . . 14
1.2.3 Policy Iteration . . . 18
1.2.4 Value Iteration . . . 26
1.2.5 Comparison . . . 31
1.2.6 The Effect of Discounting . . . 32
1.3 Generalization . . . 33

1 Dynamic Programming

Our goal is to **find an optimal policy** for an MDP. A straightforward **brute-force approach** would be to enumerate all possible deterministic policies, evaluate each one, and simply select the best. Unfortunately, this quickly becomes **impractical**. If an MDP has $|\mathcal{S}|$ states and $|\mathcal{A}|$ actions available in every state, then a deterministic policy must choose one of the actions independently for each of the states. Therefore, the total number of possible deterministic policies considered by the brute-force approach is:

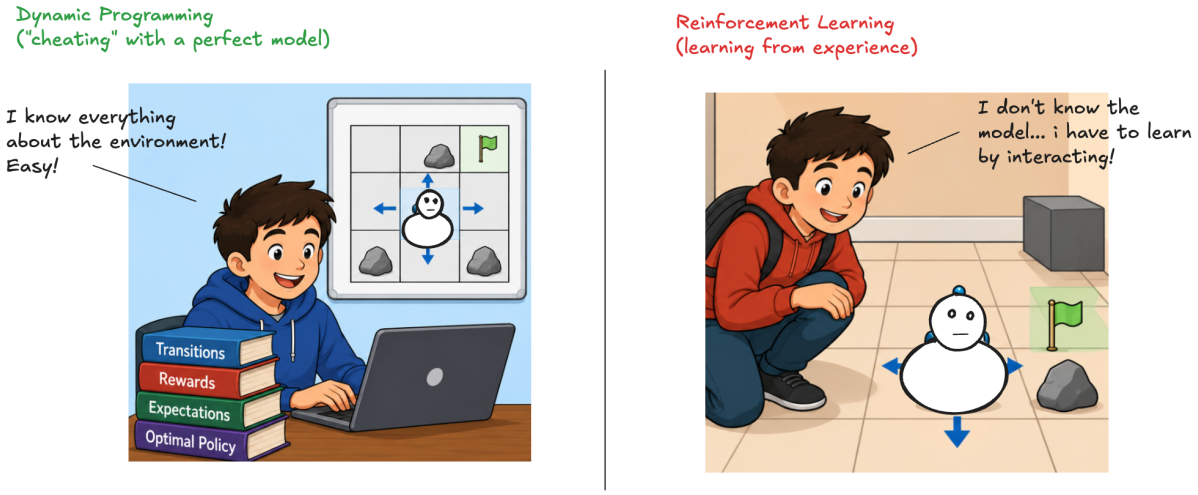
$$N_{\text{BF}} = |\mathcal{A}|^{|\mathcal{S}|}$$

The number of policies grows exponentially with the number of states. Even a relatively small MDP can have an enormous policy space, making exhaustive search infeasible. For example, with 16 states and 4 actions per state, the simple FrozenLake environment already has more than 4 billion deterministic policies:

$$N_{\text{BF}} = 4^{16} = 4.294.967.296$$

Fortunately, **sequential decision problems** have **structure** that we can exploit. The key intuition is that we do not need to solve the entire problem again for every possible policy. Different policies and trajectories often lead to the same states, and once the agent reaches the same state, it faces the same decision problem from that point onward. Why solve that future problem again and again? Instead, we can **break the original problem into smaller subproblems** associated with individual states, solve them, and reuse their solutions whenever those states are encountered. By combining these smaller solutions, we can progressively solve the complete decision

problem. This idea of solving once and reusing many times is at the heart of Dynamic Programming (DP). There is, however, an important requirement. DP assumes that we have **a perfect model of the environment**: we know the possible consequences of every action, their probabilities, and the associated rewards. In this sense, DP is somewhat "cheating". Because the complete MDP is known, we do not need to discover how the environment behaves by interacting with it. We can reason directly about all possible outcomes using the model. Consequently, there is **no need for exploration** or **trial-and-error learning**.



This assumption is restrictive because **an exact model is rarely available for real-world problems**. Nevertheless, DP is fundamental to reinforcement learning. It gives us the core ideas behind policy evaluation, policy improvement, policy iteration, and value iteration. Later, reinforcement learning will build on these same ideas while progressively removing the assumption that the environment dynamics are known, replacing model-based computations with information obtained from experience.

1.1 Value functions

Value functions tell us how good it is for an agent to be in a state, or to take a particular action in that state, in terms of the expected future return.

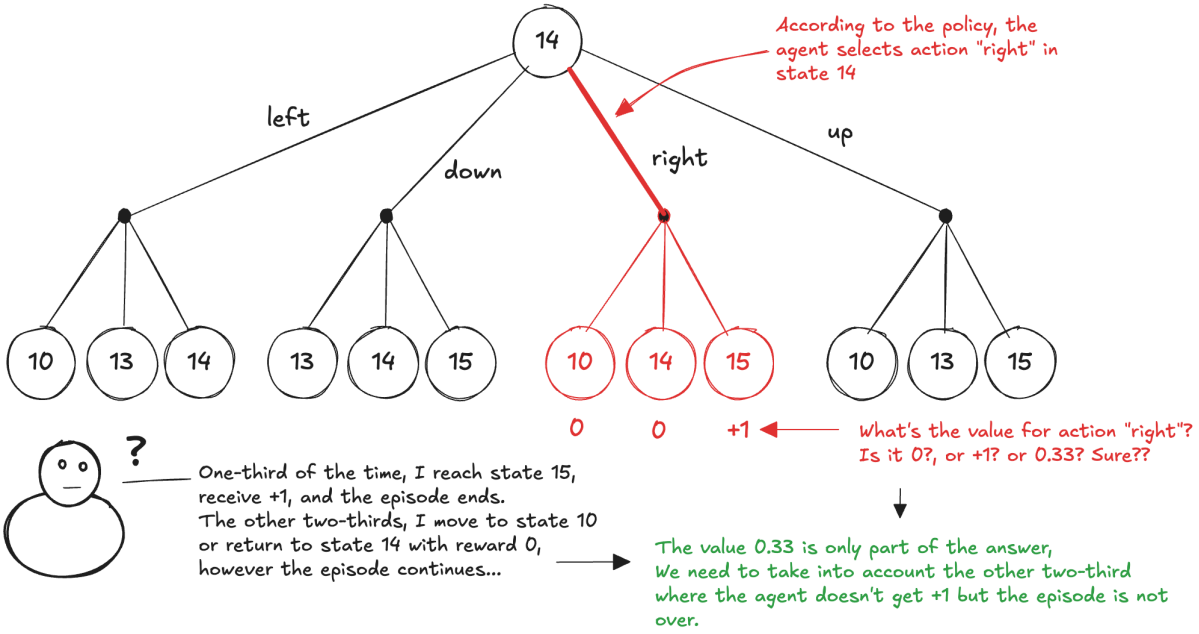
1.1.1 State-value function

For a given policy, we can **associate a value with each state** that tells us **how good it is to be in that state while following that policy**. This information can then help the agent make decisions: intuitively, we would prefer actions that move the agent toward states with higher values. To assign such a value, we consider the return obtained when starting from that state and then

following the policy. More precisely, the **state-value function** (or **V-function**) of a policy is the expected return when the agent starts in a state and follows the policy thereafter:

$$v_\pi(s) = \mathbb{E}_\pi [G_t \mid S_t = s]$$

Why do we need an **expectation**? Because in a stochastic environment, starting from the same state and following the same policy does not necessarily produce the same trajectory. The environment may respond differently to the same actions, and a stochastic policy may itself select different actions. Therefore, different trajectories (and consequently different returns) are possible. The value **summarizes all these possible futures** into a single number, weighting each possible return by its probability. In other words, it answers the question: "If I am in state and follow a policy from now on, how much return should I expect to obtain?" For example, if we follow the Go-get-it policy starting from state 14, we obtain the following value:



The calculation is not straightforward because it depends on the values of other states—states, which are also unknown. Even more interestingly, the value of one state depends on itself! We can make the recursive nature of the state-value function explicit by starting from the recursive definition of the return:

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s]$$

Applying the law of total expectation over the possible actions:

$$v_\pi(s) = \sum_a \pi(a \mid s)\, \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a]$$

We can now apply the law of total expectation again, this time over all possible next states and rewards:

$$v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a, S_{t+1} = s', R_{t+1} = r]$$

After the transition, the immediate reward is simply r, and by the Markov property, the expected future return depends only on the new state s', not on the previous state and action. Therefore,

$$v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma \mathbb{E}_\pi[G_{t+1} \mid S_{t+1} = s']\right]$$

acknowledging that

$$\mathbb{E}_\pi[G_{t+1} | S_{t+1} = s'] = v_\pi(s')$$

We obtain the **Bellman equation** for the state-value function:

$$\boxed{v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_\pi(s')\right]}$$

The key idea is that the value of a state can be expressed recursively in terms the expected immediate reward plus the discounted expected value of the next state.

1.1.2 Action-value function

Knowing how good it is to be in a state is useful, but for decision making we ultimately need to know **how good it is to take a particular action in that state**. The **action-value function** (or **Q-function**) of a policy gives the expected return obtained by taking an action in a state and then following the policy thereafter:

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$$

Using the same recursive reasoning as for the state-value function, we can derive the Bellman equation for q_π:

$$\boxed{q_\pi(s, a) = \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma \sum_{a'} \pi(a' \mid s') q_\pi(s', a')\right]}$$

The intuition is straightforward: after taking action a in state s, the environment produces a reward r and a new state s'. From s', the agent selects its next action a' according to the policy. Thus, the value of (s,a) is the expected immediate reward plus the discounted expected value of the next state-action pair.

1.2 Optimal Policies

Policies and value functions provide the tools we need to describe, evaluate, and improve behavior. The ultimate goal is to find the best possible behavior: a policy that maximizes the expected return. Value functions allow us to define **an ordering over policies**. We say that a

policy is better than or equal to another policy if its expected return is at least as large from every state:

$$\pi \geq \pi' \quad \iff \quad v_\pi(s) \geq v_{\pi'}(s) \quad \forall s \in \mathcal{S}$$

Using this order, we can conclude that always exists at least one policy that is better than or equal to every other policy. Such a policy is called an **optimal policy**:

$$\pi_* \geq \pi \quad \forall \pi$$

At first, this may seem surprising. Two policies can be incomparable: one may perform better from some states, while the other performs better from others. So how can there be a single policy that is best from every state? The key is that a policy does not make one global choice for the entire MDP. Instead, it specifies what to do in each individual state. Therefore, a policy can behave like one policy in some states and like another policy in others. In this sense, **an optimal policy can combine the best state-dependent decisions into a single policy**: whenever the agent reaches a state, it chooses an action that leads to the best possible return from that state onward. This is why, even though two arbitrary policies may not be comparable, there can still exist a policy that is at least as good as every other policy from every state. There may be **more than one optimal policy**, but all optimal policies share the same optimal state-value function, defined as the maximum value achievable from each state over all possible policies:

$$v_*(s) = \max_\pi v_\pi(s) \quad \forall s \in \mathcal{S}$$

Similarly, the optimal action-value function gives the maximum expected return achievable after taking action a in state s and behaving optimally thereafter:

$$q_*(s,a) = \max_\pi q_\pi(s,a) \quad \forall s \in \mathcal{S},\, a \in \mathcal{A}(s)$$

If we knew the optimal state-value function and the dynamics of the MDP, we could evaluate each action using a one-step lookahead:

$$q_*(s,a) = \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a]$$

Conversely, if we knew the optimal action-value function, we would no longer need a model of the environment to determine the best action. The optimal value of a state is simply the value of its best action:

$$v_*(s) = \max_a q_*(s,a)$$

Therefore, an optimal policy can be obtained directly by selecting an action that maximizes the optimal action-value function:

$$\pi_*(s) \in \operatorname*{argmax}_a q_*(s,a)$$

This is one reason why the action-value function is so important: if we know it, **optimal decision making requires neither a model of the environment nor further lookahead**.

Optimal value functions also satisfy recursive relationships. Starting from the optimal state-value function:

$v_*(s) = \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a]$

Using the dynamics of the MDP, this becomes:

$$\boxed{v_*(s) = \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_*(s')\right]}$$

The intuition is simple: for each possible action, we compute the expected immediate reward plus the discounted optimal value of the next state, and then select the action with the largest expected value. Similarly, the optimal action-value function satisfies:

$$\boxed{q_*(s,a) = \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma \max_{a'} q_*(s', a')\right]}$$

These are the **Bellman optimality equations**. Unlike the Bellman equations for a fixed policy, they do not average over actions according to a policy. Instead, they use a **maximum over actions**, because we assume that the agent behaves optimally. The Bellman optimality equations form a system of nonlinear equations with one equation for each state, or state-action pair. If the dynamics are known, then, in principle, **these equations can be solved** to obtain the state-values or action-values and therefore an optimal policy. This solution relies on at least three assumptions that are rarely true in practice:

- we accurately know the dynamics of the environment;
- we have enough computational resources to complete the computation of the solution;
- the Markov property.

Generally various combinations of these assumptions are violated and in reinforcement learning typically we have to settle for approximate solutions.

The next question is practical: **how can we actually find an optimal policy**? Dynamic programming provides a constructive answer when the MDP is fully known. The key idea is to alternate between two fundamental operations: **evaluation** and **improvement**.

1.2.1 Policy Evaluation

In order to search for good policies, we need an algorithm for evaluating an arbitrary policy more efficiently than the brute-force approach. Such an algorithm is known as a **policy evaluation** and consists of calculating the state-value function for a given policy by sweeping through the state space and iteratively improving estimates. The initial approximation V_0 can be chosen arbitrarily; a common choice is to start from zero:

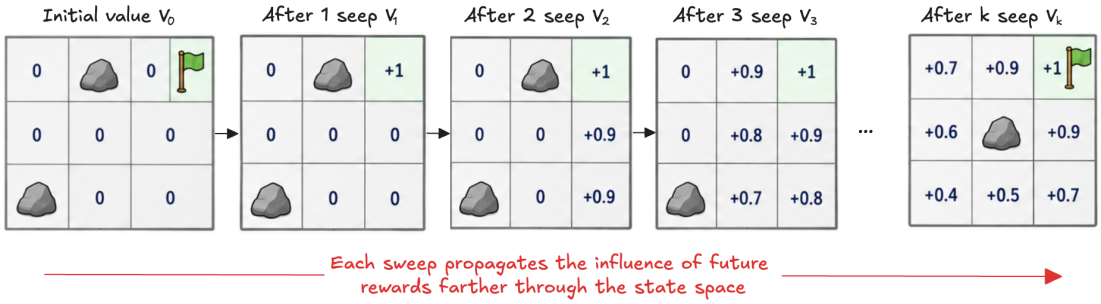
$V_0(s) = 0 \quad \forall s \in \mathcal{S}$

then each successive approximation is obtained by using the Bellman equation as an **update rule**:

$$V_{k+1}(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma V_k(s') \right]$$

Each iteration performs **a sweep through the state space**. For every state s, its current value is replaced by a new value computed from the immediate rewards and the current values of all possible successor states, weighted by the probabilities induced by the policy and the environment dynamics. An important feature of this update rule is that the new value of a state is computed using the current values of its successor states. In other words, **we improve one approximation using other approximations**. This general idea is called **bootstrapping**. This produces a sequence of approximations:

$V_0 \to V_1 \to V_2 \to \cdots$



Under the usual conditions for finite MDPs, we can prove that **this sequence converges to the true state-value function**. To understand why, it is useful to look at a policy-evaluation sweep in a slightly different way. A complete sweep takes a value function V as input, applies the Bellman expectation update to every state, and produces a new value function. We can therefore think of the Bellman update as an **operator**: simply a function that takes one value function and returns another one. For a fixed policy π, we denote this operator by T_π:

$$(T_\pi V)(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma V(s') \right]$$

With this notation, one complete policy-evaluation sweep is simply:

$V_{k+1} = T_\pi V_k$

In other words, we start from an estimation of the state value function, apply the Bellman operator, and obtain the next approximation. Now consider what happens if we apply the operator to the true value function. Since it already satisfies the Bellman expectation equation, applying the Bellman update does not change it:

$T_\pi v_\pi = v_\pi$

A value that is left unchanged by an operator is called a **fixed point** of that operator. Thus, saying that the true state value function is a fixed point of the Bellman operator simply means that once policy evaluation has reached the correct value function, another Bellman sweep leaves it unchanged. The remaining question is more important: why should repeatedly applying the Bellman operator bring an arbitrary initial value function closer and closer to this fixed point? The answer is that the Bellman operator has a useful property: when it is applied to two different value functions, their difference becomes smaller. To measure how different two value functions V and U are, we consider their largest difference over all states:

$$\|V - U\|_\infty = \max_{s \in \mathcal{S}} |V(s) - U(s)|$$

This is called the **maximum norm** or supremum norm. Intuitively, it measures the worst-case difference between the two value functions. Now apply one Bellman sweep to both V and U, their difference at a particular state s is:

$$(T_\pi V)(s) - (T_\pi U)(s)$$

Using the definition of the Bellman operator:

$$(T_\pi V)(s) - (T_\pi U)(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \left[(r + \gamma V(s')) - (r + \gamma U(s')) \right]$$

The immediate reward r is the same in the two Bellman updates, because both value functions are evaluated under the same policy and the same environment dynamics. Therefore, the reward terms cancel:

$$(T_\pi V)(s) - (T_\pi U)(s) = \gamma \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \left[V(s') - U(s') \right]$$

Taking the absolute value:

$$|(T_\pi V)(s) - (T_\pi U)(s)| = \left| \gamma \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \left[V(s') - U(s') \right] \right|$$

Using the triangle inequality (the absolute value of a sum is no greater than the sum of the absolute values):

$$|(T_\pi V)(s) - (T_\pi U)(s)| \leq \gamma \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) |V(s') - U(s')|$$

By definition of the maximum norm:

$$|V(s') - U(s')| \leq \|V - U\|_\infty$$

Therefore, since this term no longer depends on the particular next state, it can be taken outside the summations:

$$|(T_\pi V)(s) - (T_\pi U)(s)| \leq \gamma \|V - U\|_\infty \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)$$

Since the policy probabilities and transition probabilities sum to one:

$$\sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) = 1$$

hence:

$$|(T_\pi V)(s) - (T_\pi U)(s)| \leq \gamma \|V - U\|_\infty$$

Considering the largest difference over all states gives:

$$\|T_\pi V - T_\pi U\|_\infty \leq \gamma \|V - U\|_\infty$$

The maximum difference between the two value functions is reduced by at least a factor after each Bellman sweep. An operator with this property is called a **contraction**: if two value functions are different before a Bellman sweep, they are guaranteed to be closer after the sweep. We can now apply this result to the current approximation and the true value function:

$$\|T_\pi V_k - T_\pi v_\pi\|_\infty \leq \gamma \|V_k - v_\pi\|_\infty$$

Since the next approximation is the application of the Bellman operator to the current one and the true value function is a fixed point of the Bellman operator:

$$V_{k+1} = T_\pi V_k$$

$$v_\pi = T_\pi v_\pi$$

we obtain:

$$\|V_{k+1} - v_\pi\|_\infty \leq \gamma \|V_k - v_\pi\|_\infty$$

This says that after one sweep, the worst-case error with respect to the true value function is at most γ times the previous error. Applying the same reasoning repeatedly gives:

$$\|V_k - v_\pi\|_\infty \leq \gamma^k \|V_0 - v_\pi\|_\infty$$

Since $\gamma < 1$, we have:

$$\gamma^k \to 0 \qquad \text{as} \qquad k \to \infty$$

Therefore, the maximum error tends to zero:

$$\|V_k - v_\pi\|_\infty \to 0$$

and hence:

$$\boxed{\lim_{k\to\infty} V_k(s) = v_\pi(s) \qquad \forall s \in \mathcal{S}}$$

This proves the convergence of iterative policy evaluation. Intuitively, each sweep propagates information about rewards farther through the state space. After one sweep, a state incorporates information about its immediate successors, after two sweeps, information can propagate from states two steps away, and so on. As this process continues, **the influence of future rewards**

is progressively incorporated into every state value, until the values become consistent with the Bellman equation and converge to the real state-value function. Although convergence is formally obtained in the limit, in practice **we stop when the values change by only a negligible amount** between successive sweeps:

$\Delta_k = \max_{s \in \mathcal{S}} |V_{k+1}(s) - V_k(s)|$

and we stop when:

$\Delta_k < \theta$

where $\theta > 0$ is a small tolerance.

The same iterative idea can also be applied to the action-value function:

$$\boxed{Q_{k+1}(s,a) = \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma \sum_{a'} \pi(a' \mid s') Q_k(s', a') \right]}$$

It is important to understand what the index k represents. It **counts iterations** of the policy-evaluation algorithm, **not interactions** with the environment. **A sweep is not an episode**, and **k is not an environment time step**. The agent is not taking actions and collecting experience: the algorithm is repeatedly performing computations using the known MDP model until the values converge.

In the following implementation, we are considering a **deterministic policy**: it specifies a single action for each state. For a stochastic policy, the update must instead consider all possible actions and weight their contributions:

```
import numpy as np

def policy_evaluation(pi, env, gamma=0.99, theta=1e-10):

    # Access the complete transition model of the environment.
    # Dynamic programming assumes that these dynamics are known.
    dynamics = env.unwrapped.P

    # Start from an arbitrary initial value function; zero is a common choice.
    prev_V = np.zeros(len(dynamics))

    # Count the number of complete state-space sweeps required for convergence.
    sweeps = 0

    # Repeatedly apply Bellman expectation updates until convergence.
    while True:
```

```
        # Initialize the value function for the current iteration
        V = np.zeros(len(dynamics))

        # Sweep through all states.
        for s in range(len(dynamics)):

            # Deterministic policy: select the action prescribed in state s.
            action = pi(s)

            # Consider every possible transition produced by this state-action pair.
            for prob, next_state, reward, done in dynamics[s][action]:

                # Bellman expectation update:
                # expected immediate reward plus discounted successor-state value.
                # No future value is added after a terminal transition.
                continuation = 0.0 if done else gamma * prev_V[next_state]
                V[s] += prob * (reward + continuation)

        # One complete sweep through the state space has been performed.
        sweeps += 1

        # Measure the largest change produced by this sweep.
        delta = np.max(np.abs(V - prev_V))

        # Stop when the value function changes by less than the chosen tolerance.
        if delta < theta:
            break

        # Use the current values as the basis for the next sweep.
        prev_V = V.copy()

    # Return the converged approximation of v_pi.
    return V, sweeps
```

We can write a simple helper function that displays the value function according to the grid layout, making the value associated with each state easier to interpret:

```
def print_state_value_function(v, env):

    # Access the environment dynamics
    dynamics = env.unwrapped.P
```

```
    # Get the grid dimensions from the environment
    n_rows, n_cols = env.unwrapped.nrow, env.unwrapped.ncol

    print("State-value function:")

    # Print one value for each state, preserving the grid layout
    for s in range(len(dynamics)):

        # A state is terminal if every possible transition from it terminates the
        ↪  episode
        is_terminal = all(
            done
            for action_transitions in dynamics[s].values()
            for _, _, _, done in action_transitions
        )

        # Display terminal states separately
        if is_terminal:
            print("X", end="\t")
        else:
            print(f"{v[s]:.2f}", end="\t")

        # Start a new line at the end of each grid row
        if (s + 1) % n_cols == 0:
            print()
```

Let's now apply policy evaluation to our two FrozenLake policies: Go-get-it and Careful:

```
import gymnasium as gym

# Create the FrozenLake environment
frozen_lake = gym.make("FrozenLake-v1")

# Define symbolic names for the four available actions
LEFT, DOWN, RIGHT, UP = range(4)

# Define the deterministic Go-get-it policy:
def go_get_pi(s):
    return {
        0: RIGHT, 1: RIGHT, 2: DOWN, 3: LEFT,
        4: DOWN, 5: LEFT, 6: DOWN, 7: LEFT,
        8: RIGHT, 9: RIGHT, 10: DOWN, 11: LEFT,
```

```
        12: LEFT, 13: RIGHT, 14: RIGHT, 15: LEFT,
    }[s]

# Define the deterministic Careful policy:
def careful_pi(s):
    return {
        0: LEFT, 1: UP, 2: UP, 3: UP,
        4: LEFT, 5: LEFT, 6: UP, 7: LEFT,
        8: UP, 9: DOWN, 10: LEFT, 11: LEFT,
        12: LEFT, 13: RIGHT, 14: RIGHT, 15: LEFT,
    }[s]

# Compute the state-value function of the Go-get-it policy:
V_go_get_it, _ = policy_evaluation(go_get_pi, frozen_lake, gamma=0.99)

# Compute the state-value function of the Careful policy:
V_careful, _ = policy_evaluation(careful_pi, frozen_lake, gamma=0.99)

# Display the two state-value functions using the FrozenLake grid layout
print("Go-get-it:")
print_state_value_function(V_go_get_it, frozen_lake)

print("\nCareful:")
print_state_value_function(V_careful, frozen_lake)
```

```
Go-get-it:
State-value function:
0.03 0.02 0.05 0.02
0.05 X 0.10 X
0.09 0.24 0.29 X
X 0.43 0.64 X

Careful:
State-value function:
0.41 0.38 0.35 0.34
0.42 X 0.12 X
0.45 0.48 0.43 X
X 0.59 0.71 X
```

As we have already seen, the Go-get-it policy performs poorly in this environment, whereas the

Careful policy achieves much higher values. But this naturally raises an important question: **can we find an even better policy?**

1.2.2 Policy Improvement

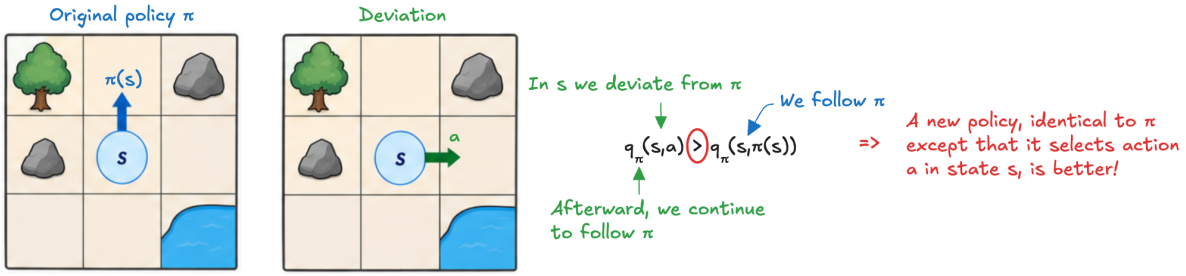
The main reason for computing the value function of a policy is to help us find a better policy. Suppose we have evaluated a policy and therefore know its state-value function. For any state s, we can now ask: **should we keep following the policy, or would it be better to choose a different action?** Suppose that, instead of taking the action prescribed by the policy, we take some different action and then follow again the policy thereafter. The expected return of doing this is:

$$q_\pi(s,a) = \sum_{s',r} p(s',r \mid s,a)\,[r + \gamma v_\pi(s')]$$

We can compare this expected return with the one obtained by following the current policy. If we find that:

$$q_\pi(s,a) > v_\pi(s)$$

then **taking the different action is better** than simply following the policy.



This intuition is formalized by the **policy improvement theorem**. Let π be the current policy and π' a new policy, then we have:

$$\boxed{q_\pi(s,\pi'(s)) \geq v_\pi(s) \quad \forall s \in \mathcal{S} \quad \Longrightarrow \quad v_{\pi'}(s) \geq v_\pi(s) \quad \forall s \in \mathcal{S}}$$

In other words, if the action selected by the new policy is at least as good as the action prescribed by the original one in every state, then the new policy is guaranteed to be at least as good as the original policy. The proof is simple:

$$q_\pi(s,\pi'(s)) \geq v_\pi(s) \quad \forall s \in \mathcal{S}$$

by definition of the action-value function,

$$q_\pi(s,\pi'(s)) = \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = \pi'(s)]$$

therefore,

$v_\pi(s) \leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s]$

now apply the same inequality again to the successor state:

$v_\pi(s) \leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 v_\pi(S_{t+2}) \mid S_t = s]$

repeating this argument recursively gives:

$v_\pi(s) \leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^k v_\pi(S_{t+k}) \mid S_t = s]$

as k grows, under the usual conditions that make the return well defined, the remaining term vanishes, and we obtain:

$v_\pi(s) \leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \mid S_t = s]$

but the quantity on the right is exactly the value of state s under the new policy:

$v_\pi(s) \leq v_{\pi'}(s)$

hence,

$v_{\pi'}(s) \geq v_\pi(s) \quad \forall s \in \mathcal{S}$

The intuition is that if the new policy always chooses an action that is at least as good as continuing with the original policy, then repeatedly following the new one cannot produce a lower expected return than following the original.

Rather than improving the policy in only one state, we can **apply the idea to every state**. For each state, we consider all possible actions and select the one with the largest action value:

$\pi'(s) = \arg\max_a q_\pi(s,a)$

Since we know the state-value function of the original policy and the MDP dynamics, we do not actually need to compute and store the complete action-value function for the improved policy, we can perform a one-step lookahead directly:

$$\boxed{\pi'(s) = \arg\max_a \sum_{s',r} p(s',r \mid s,a)\,[r + \gamma v_\pi(s')]}$$

The new policy is said to be **greedy** with respect to the state-value function of the original policy. The process of constructing such a policy from the value function of the current policy is called **policy improvement**:

```python
def policy_improvement(V, env, gamma=0.99):

    # Access the complete transition model of the environment.
    # Dynamic programming assumes that these dynamics are known.
    dynamics = env.unwrapped.P
```

```
    # Initialize the action-value function Q(s,a) to zero.
    n_states = len(dynamics)
    n_actions = env.action_space.n
    Q = np.zeros((n_states, n_actions))

    # Compute Q(s,a) for every state-action pair using one-step lookahead.
    for s in range(n_states):
        for a in range(n_actions):

            # Consider all possible transitions from state s under action a.
            for prob, next_state, reward, done in dynamics[s][a]:

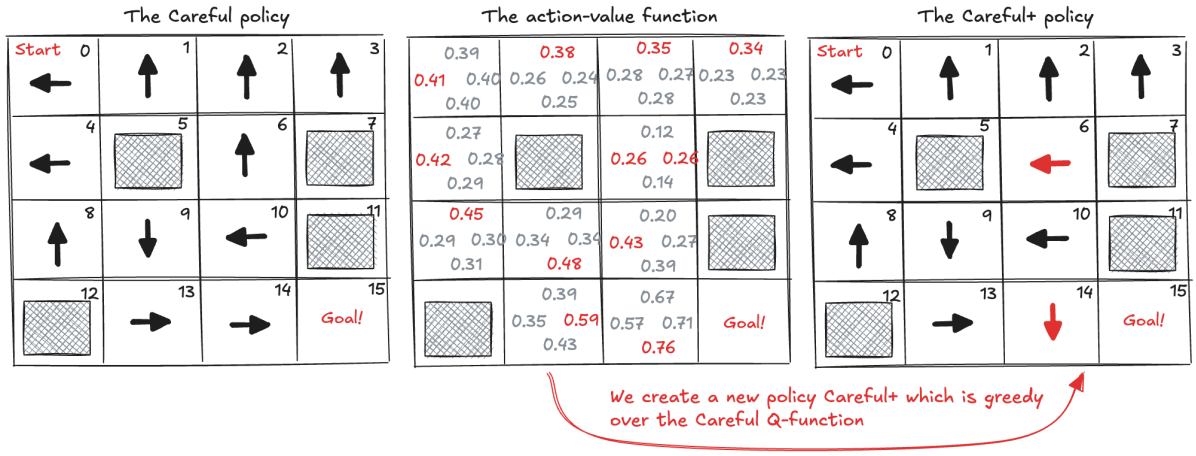
                # Expected immediate reward plus the discounted value
                # of the successor state. Terminal transitions have no future value.
                continuation = 0.0 if done else gamma * V[next_state]
                Q[s, a] += prob * (reward + continuation)

    # Construct a new policy that is greedy with respect to Q.
    def new_pi(s):
        return np.argmax(Q[s])

    # Return the improved policy.
    return new_pi
```

Let's apply one policy improvement step to the Careful policy in order to obtain a new policy, which we call Careful+:

```
careful_plus_pi = policy_improvement(V_careful, frozen_lake, gamma=0.99)
```



We can verify the effect of the improvement step more directly by comparing the state-value functions of the two policies. According to the policy improvement theorem, Careful+ should have a value greater than or equal to Careful in every state:

```
# Evaluate the improved Careful+ policy.
V_careful_plus, _ = policy_evaluation(careful_plus_pi, frozen_lake, gamma=0.99)

# Display the state-value function of the original Careful policy.
print("Careful:")
print_state_value_function(V_careful, frozen_lake)

# Display the state-value function after policy improvement.
print("\nCareful+:")
print_state_value_function(V_careful_plus, frozen_lake)

# Compute the improvement in value for each state.
difference = V_careful_plus - V_careful

# Display how much each state value has increased (or decreased).
print("\nDifference (Careful+ - Careful):")
print_state_value_function(difference, frozen_lake)
```

```
Careful:
State-value function:
0.41 0.38 0.35 0.34
0.42 X 0.12 X
0.45 0.48 0.43 X
X 0.59 0.71 X

Careful+:
State-value function:
0.54 0.50 0.47 0.46
0.56 X 0.36 X
0.59 0.64 0.62 X
X 0.74 0.86 X

Difference (Careful+ - Careful):
State-value function:
0.13 0.12 0.12 0.11
0.14 X 0.24 X
```

```
0.15 0.16 0.18 X
X 0.15 0.15 X
```

The new policy is better than the original one. This is great, acting greedily with respect to the action-value function gave us an improved policy. This is what the policy-improvement algorithm does: **it calculates an action-value function using the current policy and the MDP, then it returns an improved policy acting greedy with respect to the action-value function of the original policy**.

1.2.3 Policy Iteration

The natural next question is: **can we do even better** than Careful+? Since Careful+ is itself a policy, we can compute its state-value function, use it to improve the policy again, and repeat the process. This suggests a simple strategy: **alternate policy evaluation and policy improvement until the policy can no longer be improved**. Let's try:

```
# Apply another policy improvement step to Careful+
# using its state-value function, obtaining Careful++.
careful_plus_plus_pi = policy_improvement(V_careful_plus, frozen_lake, gamma=0.99)
```

We can now evaluate Careful++ and compare its state-value function with that of Careful+ to determine whether this additional policy improvement step produces any further improvement:

```
# Evaluate the improved Careful++ policy.
V_careful_plus_plus, _ = policy_evaluation(careful_plus_plus_pi, frozen_lake,
 ↪  gamma=0.99)

# Display the state-value function of the Careful+ policy.
print("Careful+:")
print_state_value_function(V_careful_plus, frozen_lake)

# Display the state-value function after policy improvement.
print("\nCareful++:")
print_state_value_function(V_careful_plus_plus, frozen_lake)

# Compute the improvement in value for each state.
difference = V_careful_plus_plus - V_careful_plus
```

```
# Display how much each state value has increased (or decreased).
print("\nDifference (Careful++ - Careful+):")
print_state_value_function(difference, frozen_lake)
```

```
Careful+:
State-value function:
0.54 0.50 0.47 0.46
0.56 X 0.36 X
0.59 0.64 0.62 X
X 0.74 0.86 X

Careful++:
State-value function:
0.54 0.50 0.47 0.46
0.56 X 0.36 X
0.59 0.64 0.62 X
X 0.74 0.86 X

Difference (Careful++ - Careful+):
State-value function:
0.00 0.00 0.00 0.00
0.00 X 0.00 X
0.00 0.00 0.00 X
X 0.00 0.00 X
```

This time, there is no further improvement. Careful+ is **already an optimal policy** for the FrozenLake environment: applying another greedy improvement step does not produce a better policy. Starting from Careful, we needed only one improvement step because **the initial policy was already quite good**. But this is not a requirement. Even if we started from a **deliberately poor policy**, we could repeatedly alternate between policy evaluation and policy improvement, generating progressively better policies until an optimal policy is reached. The following policy tends to move the agent away from the goal and toward the holes:

```
# Define a deliberately poor policy
def poor_pi(s):
    return {
        0:UP, 1:UP, 2:UP, 3:UP,
        4:UP, 5:LEFT, 6:UP, 7:LEFT,
```

```
        8:LEFT, 9:LEFT, 10:LEFT, 11:LEFT,
        12:LEFT, 13:LEFT, 14:LEFT, 15:LEFT
    }[s]
```

To see this process in action, let's start from the poor policy and repeatedly alternate between policy evaluation and policy improvement:

```
# Iteration 1: evaluate the initial poor policy and improve it
V_1, _ = policy_evaluation(poor_pi, frozen_lake, gamma=0.99)
poor_pi_2 = policy_improvement(V_1, frozen_lake, gamma=0.99)

# Iteration 2: evaluate the new policy and improve it again
V_2, _= policy_evaluation(poor_pi_2, frozen_lake, gamma=0.99)
poor_pi_3 = policy_improvement(V_2, frozen_lake, gamma=0.99)

# Iteration 3
V_3, _ = policy_evaluation(poor_pi_3, frozen_lake, gamma=0.99)
poor_pi_4 = policy_improvement(V_3, frozen_lake, gamma=0.99)

# Iteration 4
V_4, _ = policy_evaluation(poor_pi_4, frozen_lake, gamma=0.99)
poor_pi_5 = policy_improvement(V_4, frozen_lake, gamma=0.99)

# Iteration 5
V_5, _ = policy_evaluation(poor_pi_5, frozen_lake, gamma=0.99)
poor_pi_6 = policy_improvement(V_5, frozen_lake, gamma=0.99)

# Iteration 6
V_6, _ = policy_evaluation(poor_pi_6, frozen_lake, gamma=0.99)
poor_pi_7 = policy_improvement(V_6, frozen_lake, gamma=0.99)

# Final step, evaluate the last obtained policy
V_7, _ = policy_evaluation(poor_pi_7, frozen_lake, gamma=0.99)
```

This procedure of repeatedly alternating policy evaluation and policy improvement is called **policy iteration**. To visualize how policy iteration progressively approaches the optimal policy, we can plot the **maximum gap** from the optimal value function after each evaluation–improvement cycle:

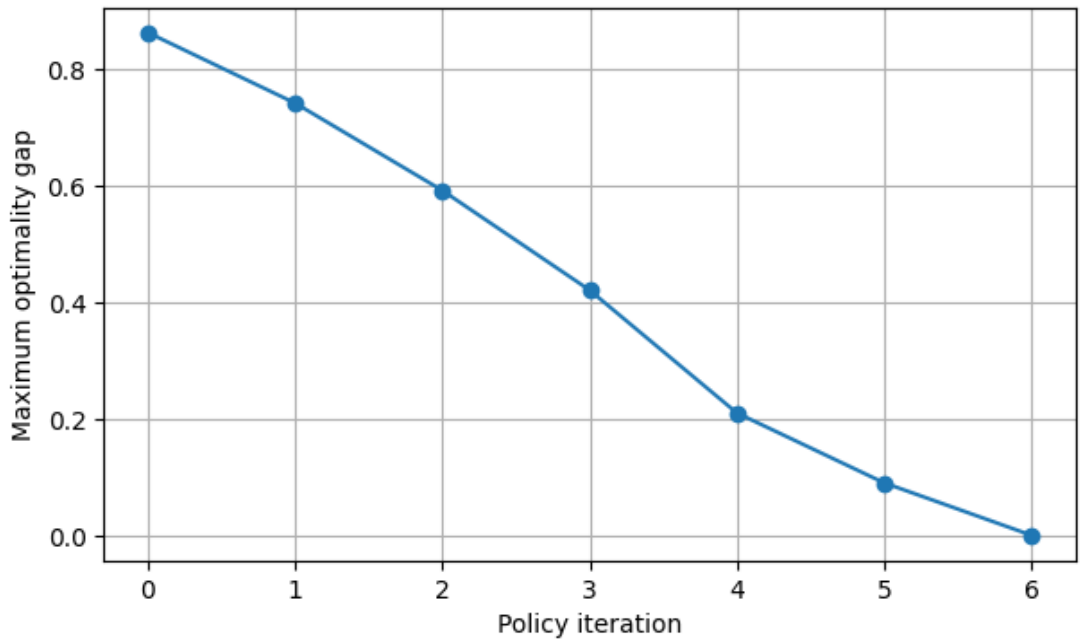
$$\text{gap}_k = \max_s |V_*(s) - V_k(s)|$$

```
import matplotlib.pyplot as plt

# Value functions obtained during policy iteration
value_functions = [V_1, V_2, V_3, V_4, V_5, V_6, V_7]

# Compute the maximum optimality gap at each iteration
optimality_gaps = [
    np.max(V_careful_plus - V)
    for V in value_functions
]

# Plot the optimality gap
plt.figure(figsize=(7, 4))
plt.plot(range(len(optimality_gaps)), optimality_gaps, marker="o")
plt.xlabel("Policy iteration")
plt.ylabel("Maximum optimality gap")
plt.xticks(range(len(optimality_gaps)))
plt.grid()
plt.show()
```



We obtain **a sequence of monotonically improving policies and value functions**:

$$\pi_0 \overset{\text{evaluate}}{\longrightarrow} v_{\pi_0} \overset{\text{improve}}{\longrightarrow} \pi_1 \overset{\text{evaluate}}{\longrightarrow} v_{\pi_1} \overset{\text{improve}}{\longrightarrow} \pi_2 \overset{\text{evaluate}}{\longrightarrow} \cdots \overset{\text{improve}}{\longrightarrow} \pi_* \overset{\text{evaluate}}{\longrightarrow} v_{\pi_*}$$

Each improvement step produces a policy that is at least as good as the previous one. Eventually,

policy improvement may produce no change at all. Why does this imply that the current policy is optimal? Suppose that the current policy is π. Policy evaluation has already computed its value function. The improvement step then examines, in every state s, all possible actions and chooses an action with the largest action value:

$\pi'(s) = \arg\max_a q_\pi(s,a)$

Now suppose that **the policy is stable**, meaning that this greedy improvement does not change it:

$\pi'(s) = \pi(s) \qquad \forall s$

This means that the action already selected by the current policy is a greedy action with respect to its own value function:

$\pi(s) \in \arg\max_a q_\pi(s,a)$

Because v_π is the value of following π, we have:

$v_\pi(s) = q_\pi(s, \pi(s))$

But stability tells us that $\pi(s)$ already maximizes $q_\pi(s,a)$ over all possible actions. Therefore:

$v_\pi(s) = \max_a q_\pi(s,a)$

Expanding the definition of q_π gives:

$v_\pi(s) = \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_\pi(s') \right]$

Now comes the crucial observation: this is not the ordinary Bellman equation for policy π. It is the Bellman optimality equation:

$v_*(s) = \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_*(s') \right]$

For a finite discounted MDP, the Bellman optimality equation has a unique solution. Since v_π satisfies exactly this equation, it must therefore be that:

$v_\pi = v_*$

and consequently the stable policy is also an optimal policy. Thus, policy iteration cannot stop at a merely locally improved policy, if no greedy improvement is possible, the current policy is already globally optimal. Since a finite MDP has only finitely many deterministic policies, and each improvement step produces a policy that is strictly better unless it is already optimal, the sequence can never revisit an earlier policy, so it is guaranteed to reach an optimal policy after a finite number of improvement steps. We can implement this idea in Python:

```python
def policy_iteration(env, gamma=0.99, theta=1e-10):

    # Access the complete transition model of the environment.
    # Dynamic programming assumes that these dynamics are known.
    dynamics = env.unwrapped.P

    # Initialize an arbitrary deterministic policy by selecting
    # one random action for each state.
    actions = tuple(dynamics[0].keys())
    random_actions = {
        s: np.random.choice(actions)
        for s in range(len(dynamics))
    }

    def pi(s):
        return random_actions[s]

    # Track the number of evaluation-improvement cycles and the total
    # number of policy-evaluation sweeps performed by the algorithm.
    total_sweeps, cycles = 0, 0

    # Alternate policy evaluation and policy improvement
    # until the policy becomes stable.
    while True:

        # Evaluate the current policy.
        V, sweeps = policy_evaluation(pi, env, gamma, theta)

        # Improve the policy by acting greedily with respect to v.
        new_pi = policy_improvement(V, env, gamma)

        # Accumulate the policy-evaluation sweeps and count this
        # evaluation-improvement cycle.
        total_sweeps += sweeps
        cycles += 1

        # If no action changes, the policy is stable and therefore optimal.
        policy_stable = all(
            pi(s) == new_pi(s)
            for s in range(len(dynamics))
        )

        if policy_stable:
```

```
            break

        # Continue with the improved policy.
        pi = new_pi

    # Return the optimal state-value function and an optimal policy.
    return V, new_pi, cycles, total_sweeps
```

As an example, we can run Policy Iteration over the Frozen Lake environment and verify that it converges to the optimal value function:

```
# Run policy iteration starting from an arbitrary policy.
V_policy_iteration, pi_policy_iteration, _, _ = policy_iteration(frozen_lake,
 ↪  gamma=0.99)

# Display the state-value function obtained by policy iteration.
print("Policy Iteration:")
print_state_value_function(V_policy_iteration, frozen_lake)

# Compare it with the optimal value function obtained from Careful+.
print("\nOptimality gap:")
print_state_value_function(V_careful_plus - V_policy_iteration, frozen_lake)
```

```
Policy Iteration:
State-value function:
0.54 0.50 0.47 0.46
0.56 X 0.36 X
0.59 0.64 0.62 X
X 0.74 0.86 X

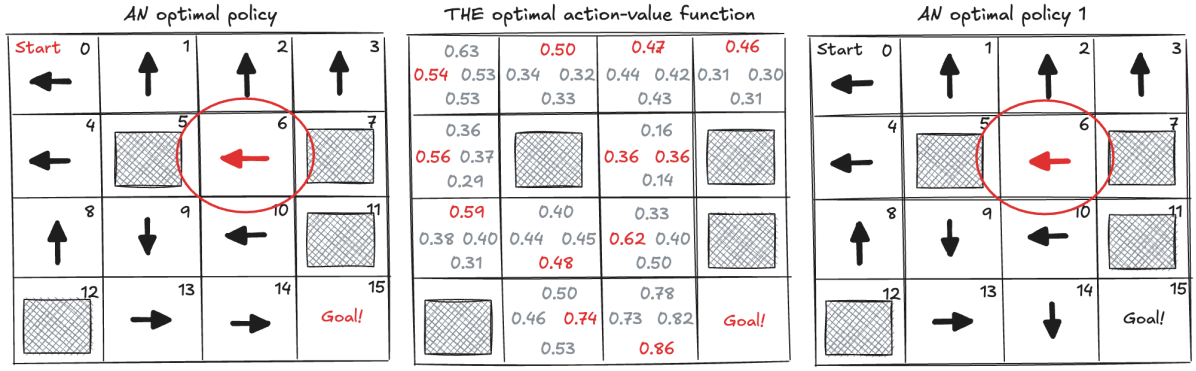
Optimality gap:
State-value function:
0.00 0.00 0.00 0.00
0.00 X 0.00 X
0.00 0.00 0.00 X
X 0.00 0.00 X
```

As expected, alternating policy evaluation and policy improvement yields an optimal policy and the optimal state-value function, regardless of the initial policy. Notice the deliberate distinction between **"an" optimal policy** and **"the" optimal state-value function**. An MDP may

have multiple optimal policies, but they all share the same unique optimal state-value function. To understand how this is possible, we can look at the optimal action-value function. Recall that:

$v(s) = \max_a q(s, a)$

If two or more actions achieve **the same maximum value in a state**, then any of them can be selected by an optimal policy.Therefore, different optimal policies may choose different actions in the same state while still producing the same optimal state value. The FrozenLake environment provides a concrete example:



In finding the optimal policy, here we have a crucial advantage over the brute-force search considered earlier: **policy iteration does not explore the policy space blindly**. Each new policy is constructed by systematically improving the previous one, producing a directed sequence of improving policies rather than evaluating arbitrary policies one by one. If policy iteration requires K improvement steps to reach an optimal policy, it considers only the (K+1) policies along this sequence:

$N_{\text{PI}} = K + 1$

In contrast, exhaustive brute-force search considers the entire space of deterministic policies:

$N_{\text{BF}} = |\mathcal{A}|^{|\mathcal{S}|}$

Thus, in the typical case:

$$\boxed{N_{\text{PI}} = K + 1 \ll N_{\text{BF}} = |\mathcal{A}|^{|\mathcal{S}|}}$$

The value of K depends on the MDP and the initial policy, and the relation above should be understood as a practical observation rather than a worst-case complexity guarantee. The fundamental advantage is that every policy considered by policy iteration is constructed using information obtained from the previous policy.

So far, we have considered the special case of deterministic policies, but the same ideas extend naturally to **stochastic policies**. In particular, when several actions achieve the same maximum value, a stochastic greedy policy does not need to choose only one of them. Instead, it can distribute the probability of selection among the maximizing actions, while assigning zero probability to all non-maximizing actions.

1.2.4 Value Iteration

Policy iteration finds an optimal policy by repeatedly solving two different problems: evaluate the current policy and then improve the policy. During policy evaluation, however, we may perform many sweeps to accurately compute the state value function, even though the current policy is only an intermediate that we are about to change. This raises a natural question: if our ultimate goal is the optimal value function, **do we really need to compute the exact value of every intermediate policy?** Recall the Bellman equations:

$$v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_\pi(s') \right]$$

Policy evaluation repeatedly applies the corresponding update until the values become consistent with that particular policy. But the quantity we ultimately want is not this state value function. We want the optimal one, which satisfies the Bellman optimality equation:

$$v_*(s) = \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_*(s') \right]$$

This suggests a **more direct strategy**: rather than repeatedly computing the value of one policy and then replacing that policy, iterate directly toward the solution of the Bellman optimality equation. Starting from an arbitrary state value function, **value iteration** repeatedly applies:

$$V_{k+1}(s) = \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma V_k(s') \right]$$

Starting from any arbitrary initial value function, it generates the sequence:

$$V_0 \to V_1 \to V_2 \to \cdots \to v_*$$

We can justify the convergence using the same argument introduced for policy evaluation. The main difference is that we now introduce the Bellman optimality operator instead of the Bellman expectation operator:

$$(T_* V)(s) = \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma V(s') \right]$$

With this notation, one complete value-iteration sweep is:

$$V_{k+1} = T_* V_k$$

The optimal state-value function satisfies the Bellman optimality equation, so applying the operator leaves it unchanged:

$T_* v_* = v_*$

Therefore, the optimal value function is a fixed point of the Bellman optimality operator. As in policy evaluation, we now show that applying the operator brings value functions closer together, so that T_* is also a contraction. The only new ingredient with respect to T_π is the maximization over actions, so we first need a simple inequality relating two maxima. For any two collections of numbers x_a and y_a indexed by the same actions:

$$\left|\max_a x_a - \max_a y_a\right| \leq \max_a |x_a - y_a|$$

Intuitively, the difference between two maxima cannot be larger than the largest difference between the quantities being maximized. Now apply one Bellman optimality sweep to both V and U. Their difference at a particular state s is:

$$(T_* V)(s) - (T_* U)(s) = \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma V(s')\right] - \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma U(s')\right]$$

This is exactly a difference of two maxima, with $x_a = \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma V(s')\right]$ and $y_a = \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma U(s')\right]$. Applying the inequality above:

$$|(T_* V)(s) - (T_* U)(s)| \leq \max_a \left| \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma V(s')\right] - \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma U(s')\right] \right|$$

For a fixed action a, the immediate reward r is the same in both terms, so it cancels, exactly as it did for T_π:

$$\sum_{s',r} p(s', r \mid s, a) \left[r + \gamma V(s')\right] - \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma U(s')\right] = \gamma \sum_{s',r} p(s', r \mid s, a) \left[V(s') - U(s')\right]$$

Using the triangle inequality, then bounding $|V(s') - U(s')| \leq \|V - U\|_\infty$ and pulling this constant out of the sum:

$$\left| \gamma \sum_{s',r} p(s', r \mid s, a) \left[V(s') - U(s')\right] \right| \leq \gamma \|V - U\|_\infty \sum_{s',r} p(s', r \mid s, a) = \gamma \|V - U\|_\infty$$

where the last equality holds because, for every action a, the transition probabilities sum to one. Since this bound no longer depends on a, it also bounds the maximum over a, and therefore:

$$|(T_* V)(s) - (T_* U)(s)| \leq \gamma \|V - U\|_\infty$$

Taking the largest difference over all states gives the contraction property of T_*:

$$\|T_* V - T_* U\|_\infty \leq \gamma \|V - U\|_\infty$$

Thus, just like the Bellman expectation operator, the Bellman optimality operator is a **contraction**: one application reduces the maximum difference between two value functions by at least a factor γ. We can now compare the current approximation with the optimal value function:

$\|T_* V_k - T_* v_*\|_\infty \leq \gamma \|V_k - v_*\|_\infty$

Since the next approximation is the application of the Bellman optimality operator to the current one and the true value function is a fixed point of the Bellman optimality operator:

$V_{k+1} = T_* V_k$

$v_* = T_* v_*$

we obtain:

$\|V_{k+1} - v_*\|_\infty \leq \gamma \|V_k - v_*\|_\infty$

Repeating the same argument over k iterations gives:

$\|V_k - v_*\|_\infty \leq \gamma^k \|V_0 - v_*\|_\infty$

Since $\gamma < 1$, we have

$\gamma^k \to 0 \qquad \text{as} \qquad k \to \infty$

Therefore, the maximum error tends to zero:

$\|V_k - v_*\|_\infty \to 0$

and hence:

$$\boxed{\lim_{k\to\infty} V_k(s) = v_*(s) \qquad \forall s \in \mathcal{S}}$$

Thus, value iteration converges to the optimal state-value function from any initial value function. Notice the strong parallel with policy evaluation:

$V_{k+1} = T_\pi V_k \longrightarrow v_\pi$

whereas

$V_{k+1} = T_* V_k \longrightarrow v_*$

The convergence mechanism is the same. In both cases, we repeatedly apply a contraction that moves the current value function toward its fixed point. The difference is which fixed point we approach: **the Bellman operator converges to the value function of a particular policy**, whereas **the Bellman optimal operator converges to the optimal value function**. As with iterative policy evaluation, exact convergence occurs only in the limit. In practice, we stop when the largest change during a sweep becomes sufficiently small:

$\Delta_k = \max_{s \in \mathcal{S}} |V_{k+1}(s) - V_k(s)| < \theta$

Only after the values have converged do we need to explicitly construct a policy. An optimal policy is obtained by choosing an action that is greedy with respect to the state value function:

$\pi_*(s) = \arg\max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_*(s') \right]$

This also explains the computational motivation for value iteration. Suppose policy evaluation requires L sweeps. Under the simplifying assumption that each state-action pair can have up to $|\mathcal{S}|$ successor states, one complete policy-iteration cycle costs approximately:

$$C_{\text{PI,cycle}} = O\left(|\mathcal{S}|^2(L + |\mathcal{A}|)\right)$$

A value-iteration sweep evaluates every action in every state once:

$$C_{\text{VI,sweep}} = O(|\mathcal{S}|^2|\mathcal{A}|)$$

Value iteration therefore avoids spending L sweeps accurately evaluating an intermediate policy before allowing another improvement. Instead, every sweep immediately contributes toward optimality. Notice that **this does not mean that value iteration must always finish faster than policy iteration**: policy iteration generally performs fewer but more expensive outer iterations, whereas value iteration generally performs more but cheaper sweeps. The important point is that value iteration eliminates the need to solve a sequence of intermediate policy-evaluation problems. We can now implement this algorithm in Python:

```python
def value_iteration(env, gamma=0.99, theta=1e-10):

    # Access the complete transition model of the environment.
    # Dynamic programming assumes that these dynamics are known.
    dynamics = env.unwrapped.P

    # Initialize V_0 arbitrarily; zero is a common choice.
    V = np.zeros(len(dynamics))

    # Count the number of complete state-space sweeps required for convergence.
    sweeps = 0

    # Repeatedly apply the Bellman optimality update until convergence.
    while True:

        # # Initialize the action value function for the current iteration
        Q = np.zeros((len(dynamics), env.action_space.n))

        # Sweep through all states and actions.
        for s in range(len(dynamics)):
            for a in range(env.action_space.n):

                # Consider all possible transitions from state s under action a.
                for prob, next_state, reward, done in dynamics[s][a]:

                    # Compute the expected return of taking action a.
```

```
                    # Terminal transitions have no future value.
                    continuation = 0.0 if done else gamma * V[next_state]
                    Q[s, a] += prob * (reward + continuation)

        # For each state, keep the value of the best available action.
        new_V = np.max(Q, axis=1)

        # Measure the largest change produced by this sweep.
        delta = np.max(np.abs(new_V - V))

        # Update the state-value function.
        V = new_V

        # One complete sweep through the state space has been performed.
        sweeps += 1

        # Stop when the values change by less than the chosen tolerance.
        if delta < theta:
            break

    # Once the values have converged, extract a greedy optimal policy.
    def pi(s):
        return np.argmax(Q[s])

    # Return the optimal state-value function and an optimal policy.
    return V, pi, sweeps
```

Once again, we can apply the algorithm to solve the FrozenLake problem:

```
# Run value iteration on the FrozenLake environment.
V_value_iteration, pi_value_iteration, _ = value_iteration(frozen_lake, gamma=0.99)

# Display the optimal state-value function obtained by value iteration.
print("Value Iteration:")
print_state_value_function(V_value_iteration, frozen_lake)

# Compare it with the optimal value function previously obtained from Careful+.
print("\nOptimality gap:")
print_state_value_function(V_careful_plus - V_value_iteration, frozen_lake)
```

```
Value Iteration:
```

```
State-value function:
0.54 0.50 0.47 0.46
0.56 X 0.36 X
0.59 0.64 0.62 X
X 0.74 0.86 X

Optimality gap:
State-value function:
0.00 0.00 0.00 0.00
0.00 X 0.00 X
0.00 0.00 0.00 X
X 0.00 0.00 X
```

As expected, value iteration converges to the same optimal state-value function as policy iteration, but it reaches it without fully evaluating an intermediate policy at each step.

1.2.5 Comparison

We can make the difference between Policy Iteration and Value Iteration concrete by running both algorithms on FrozenLake and comparing not only their execution time, but also the number of complete state-space sweeps they require to reach convergence.

```python
import time

# Run Policy Iteration and measure its execution time.
t0 = time.perf_counter()
_, _, pi_cycles, pi_sweeps = policy_iteration(frozen_lake, gamma=0.99)
t_pi = time.perf_counter() - t0

# Run Value Iteration and measure its execution time.
t0 = time.perf_counter()
_, _, vi_sweeps = value_iteration(frozen_lake, gamma=0.99)
t_vi = time.perf_counter() - t0

# Report the computational effort required by the two algorithms.
print(f"Policy Iteration: {pi_cycles} evaluate-improve cycles, {pi_sweeps} total
↪  sweeps, {t_pi*1000:.1f} ms")
print(f"Value Iteration:  {vi_sweeps} total sweeps, {t_vi*1000:.1f} ms")
```

```
Policy Iteration: 4 evaluate-improve cycles, 1250 total sweeps, 30.9 ms
```

```
Value Iteration:  571 total sweeps, 21.3 ms
```

Policy iteration performs **relatively few but expensive improvement cycles**: each policy is evaluated almost to convergence before being improved. Value iteration performs **cheaper, more frequent improvements**: every sweep immediately incorporates the best action according to the current values. Both converge to the same optimal solution; they differ mainly in how tightly evaluation and improvement are interleaved.

1.2.6 The Effect of Discounting

So far, we have used a discount factor of 0.99. But how does the choice of this factor affect the optimal value function, and potentially the optimal policy? Recall that the discount factor determines **how much future rewards matter** compared with immediate rewards. When the factor is close to 1, rewards received far in the future retain most of their value. As it decreases, delayed rewards become progressively less valuable:

$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$

FrozenLake provides a useful example because the agent receives a positive reward only when it reaches the goal. With no discounting, reaching the goal after a long trajectory is just as valuable as reaching it quickly. With discounting instead, the same reward becomes less valuable the longer it takes to reach the goal. Therefore, **discounting introduces a preference not only for reaching the goal, but also for reaching it sooner**.

Let's solve FrozenLake again with no discounting and with a stronger discount, and compare the resulting optimal value functions and policies:

```
# Compute the optimal state-value function with no discounting (gamma = 1).
V_no_discounting, _, _ = value_iteration(frozen_lake, gamma=1.00)

# Compute the optimal state-value function with stronger discounting (gamma = 0.95).
V_discounting, _, _ = value_iteration(frozen_lake, gamma=0.95)

# Display the two value functions for comparison.
print("No discounting (gamma = 1.00):")
print_state_value_function(V_no_discounting, frozen_lake)

print("\nDiscounting (gamma = 0.95):")
print_state_value_function(V_discounting, frozen_lake)
```

```
No discounting (gamma = 1.00):
```

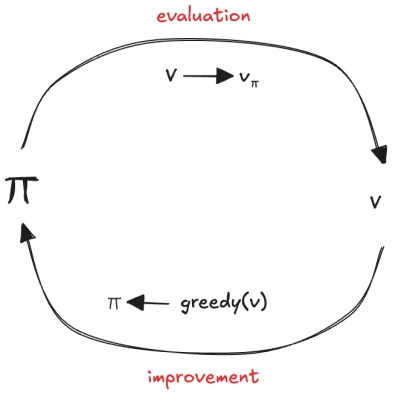
```
State-value function:
0.82 0.82 0.82 0.82
0.82 X 0.53 X
0.82 0.82 0.76 X
X 0.88 0.94 X

Discounting (gamma = 0.95):
State-value function:
0.18 0.15 0.15 0.13
0.21 X 0.18 X
0.27 0.37 0.40 X
X 0.51 0.72 X
```

We find that the resulting optimal value functions are not the same. In a stochastic environment, introducing discounting may also change the optimal policy: because delayed rewards are worth less, **discounting can favor faster but potentially riskier paths over safer paths that take longer to reach the goal**.

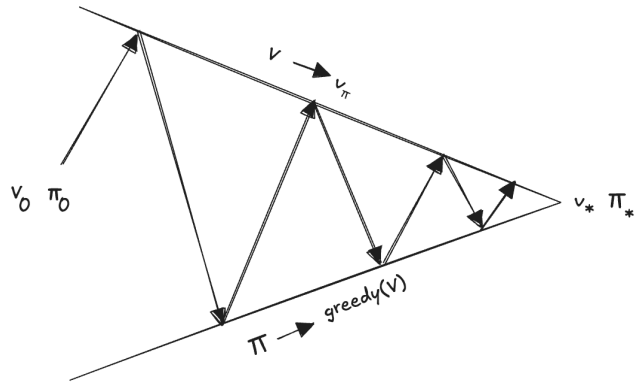
1.3 Generalization

Dynamic programming can be understood as the interaction of **two fundamental processes**: policy **evaluation**, which makes the value function increasingly consistent with the current policy, and policy **improvement**, which makes the policy increasingly greedy with respect to the current value function:



These two processes can interact at different **levels of granularity**. In policy iteration, evaluation is allowed to converge before the improvement. In value iteration, evaluation is truncated after a single sweep and the improvement is incorporated. They can also be interleaved at an

even finer level. The general principle does not depend on exactly how the two processes are organized. This broader idea is called **Generalized Policy Iteration** (**GPI**): the value function is continually adjusted toward the value function of the current policy, while the policy is continually improved with respect to the current value function. The two processes can be viewed as **simultaneously cooperating and competing**. When we improve the policy, the current value function is no longer fully consistent with the new policy and must be updated. Conversely, as we update the value function, new information may reveal that the current policy is no longer greedy with respect to it, creating an opportunity for further improvement. Thus, evaluation and improvement continually drive each other:



If both processes eventually stabilize, neither the value function nor the policy can be further improved: the value function is the optimal value function and the policy is an optimal policy. GPI is much more general than dynamic programming. It provides a useful conceptual framework for understanding a large fraction of reinforcement learning algorithms: values guide the improvement of behavior, while the resulting behavior determines the values that should be learned or computed.